

A Framework for Personalized Recommendation Based on LLMs with Constrained Combinatorial Optimization

Naoki Ishii

Department of Mathematical Informatics / Mathematical
Informatics 3rd Laboratory
The University of Tokyo
Tokyo, Japan
Preferred Networks Inc.
Tokyo, Japan
ishii-naoki@g.ecc.u-tokyo.ac.jp

Keita Higuchi

Preferred Networks Inc.
Tokyo, Japan
khiguchi@acm.org

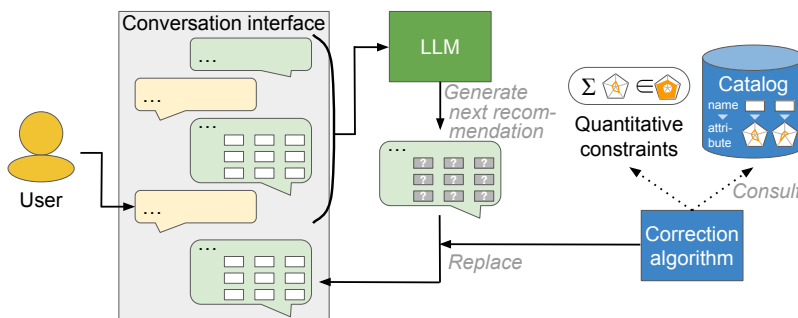


Figure 1: The overall concept of the proposed framework. The process that occurs between receiving a new user remark and presenting a response to the user is depicted. First, the LLM generates a draft response based on the whole conversation history. Then, the “correction algorithm” module modifies the recommendation in the draft so that it consists of items in the catalog and satisfies the user-provided constraints. Left-aligned yellow bubbles are texts input by the user. Right-aligned green bubbles are texts output by the system. The gray items in the LLM’s response cannot necessarily be found in the catalog. The white items in the system’s actual responses in the conversation are from the catalog.

Abstract

This work proposes a personalized recommendation framework that satisfies individual user preferences and quantitative constraints through conversation with a large language model (LLM). Recent studies have applied LLMs to personalized recommendations, but they sometimes recommend non-existent items (hallucination) or cannot strictly adhere to user-specific constraints. The proposed framework replaces recommended items from LLMs with existing items to prevent hallucination. The recommender satisfies user constraints expressed as inequalities by a combination set of recommended items. We applied our framework to a diet recommendation system that outputs a list of dishes to eat during a day through three meals, considering prompts and nutritional goals of users. In our user study, all of the recommendations from our

framework satisfied specified nutritional goals. The dishes recommended by our framework reflected user preferences at the same level as a baseline which ignores nutritional constraints and recommends the closest existing dishes to those in the LLM’s response. Our framework extends the use cases of LLM-based conversational interfaces to tasks with quantitative constraints.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI)**; • **Information systems** → **Recommender systems**; **Personalization**.

Keywords

Large Language Model, Personalized Recommendation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3719960>

ACM Reference Format:

Naoki Ishii and Keita Higuchi. 2025. A Framework for Personalized Recommendation Based on LLMs with Constrained Combinatorial Optimization. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3706599.3719960>

1 Introduction

The recent development of large language models (LLMs) has drastically changed the landscape of natural language processing (NLP). It has made it possible to create high-quality “chatbots”, or user interfaces in conversational interaction in natural language, such as ChatGPT [17] and Gemini [27]. LLMs can generate a reply text based on the given prompt without fine-tuning. This ability is called *in-context learning* [4, 28]. Taking advantage of this, recent research on knowledge-based recommender systems has tried to use LLMs for personalized recommendation tasks by modeling user preferences through human–LLM interaction [1].

LLM-powered recommendation, however, embraces two challenges. First, in recommendation systems that suggest an item or a set of items from a predefined finite set (catalog-based recommendation), LLM hallucination may cause recommendation of nonexistent items. Besides, the limit of the prompt length prevents the model from being instructed about the set. Second, under circumstances where qualitative constraints are imposed by users, it is unrealistic for an LLM to find a valid recommendation since they have no capability of conducting advanced combinatorial algorithms.

To address these difficulties, we propose a recommendation scheme that satisfies both user preference and constraints by combining an LLM with an external algorithmic module (Figure 1). The LLM serves to capture the user preference. For each response in the conversation with the user, the model’s recommendation is used as a draft, and the item names in it are then replaced with those from the predefined catalog. This replacement guarantees satisfaction of the constraints. To locate the names of recommended items to replace in the LLM’s output, we use a prompt to make the LLM enclose the names of items with markers in the generated text. The algorithmic module finds a constraint-abiding set of items that is semantically similar to the LLM’s recommendation. It is based on hill climbing and dynamic programming and is computationally inexpensive enough to be used in a chat application. This framework covers various situations including customized commerce under a budget constraint and diet planning under nutrition conditions.

We applied the method to a diet recommendation system that generates a set of daily recipes based on prompts of user requests and individual nutritional conditions. The system suggests a few dishes for each of the three meals from a catalog with more than 4,000 recipes and accompanying nutritional information. It recommends combinations of 7–9 recipes that adhere to the recommended intake levels of essential nutrients in individual users’ diets for their health, including carbohydrates, proteins, fats, and sodium. With the system, users can receive recommendations that both align with their preferences and meet nutritional constraints, within approximately 10 seconds of users’ input of their preferences via chat.

We performed a user study to compare the proposed method with the baseline using a graphical user interface. As a result, except for cases where the system generated no output in the valid format, the proposed method always suggested meal plans that satisfy individual nutrient constraints. Also, the proposed method showed comparable performance with the baseline in reflecting participants’ preferences. Based on the results, we discussed the challenges and improvements of the proposed method.

2 Related Work

Personalized recommendation systems address the problem of suggesting relevant information or items tailored to individual users in several applications such as shopping services [12, 25], music recommendations [20, 31] and image enhancement [9, 11]. In the context of travel route recommendation, several studies considered personal preference with a budget [3, 30]. Ozeki et al. proposed a personalized recommendation method for satisfying both user preference and requirements that considers ingredient preferences and daily nutrient intakes in a diet planning application [18]. Their system recommends recipes, taking into account both the availability of ingredients specified by the user and the fulfillment of nutritional constraints. However, the system requires users to invest time in inputting their desired ingredient list to receive recipe recommendations, and furthermore, the recommended meal plans may not guarantee strict adherence to nutritional constraints. In this study, we propose a recommendation method that allows users to input their preferences in natural language via chats and ensures strict adherence to the constraint conditions.

Recent studies have focused on leveraging LLMs for recommender systems [6]. Zhang et al. created a benchmark for evaluating the fairness of LLM-based recommendation [32]. They reported the challenges for fairness and the concerns of hallucinations in generated recommendations. Recent studies also focused on LLM-based embedding for vector representations of recommended target items [10, 21]. Wu et al. utilize LLMs to convert user-profiles and job information into vector representations within the context of online job recommendation tasks. Their method performs matching by calculating vector similarities to recommend personalized job information [29]. In this paper, we utilize LLMs both for response generation and for giving vector representations of items. Our approach is to create a draft recommendation using an LLM and then change it into one that consists of items from the catalog and satisfies the user’s constraints. The modification is based on the metric of items given by vector representations computed by another LLM. As a result, it can generate meal plans in real-time, considering over 4,000 recipes as potential recommendations while adhering to nutritional constraints and meeting the user’s preferences.

3 Proposed Method: Linearly Constrained Recommendation via LLM-powered Conversation with Correction

3.1 Overview of the general framework

3.1.1 Problem description. The task our proposed method addresses is to recommend a set of items that matches *user preference* and satisfies *user constraints*. The user preference is implicitly given through a conversational interface. The user constraints are explicitly given based on quantitative attributes associated with recommended items. For example, the user may give the budgets as constraints in a shopping recommendation problem where prices are associated with items, or impose nutritional goals in a diet recommendation problem where nutritional content is associated with items.

In addition, we make several assumptions. The set of all nonnegative integers is hereinafter denoted by $\mathbb{Z}_{\geq 0}$.

- (1) We have a predefined database that contains a set of items C . In this paper, C is called *catalog* and elements of C *recommendable items*. For each recommendable item $c \in C$, the database contains a vector v_c that represents the qualitative attributes of c . All vectors v_c have the same dimension d , are nonzero vectors, and only have nonnegative integer entries. In short, $v_c \in \mathbb{Z}_{\geq 0}^d \setminus \{0\}$. We only use items in C for recommendations.
- (2) All user constraints imposed are represented by an expression $\sum_{c \in C} v_c \in \Omega$ using a bounded region $\Omega \subset \mathbb{Z}_{\geq 0}^d$. A set $X \subset C$ that satisfies the user constraints $\sum_{c \in X} v_c \in \Omega$ is hereinafter called a **feasible solution**.
- (3) The LLM we use has the strong enough ability of in-context learning to generate in the conversation with the user a hallucinatory recommendation (set of names of items that are not necessarily found in C) that aligns with the user preference, although it may be ignorant of the user constraints.
- (4) Each recommendable item $c \in C$ is associated with its name. Given a name of an item regardless of whether it is in C or hallucinatory, we can compute its *vector embeddings* (a natural language processing technique to express the meaning of a sentence as a vector). There are various models and learning methods to compute vector embeddings of words or sentences [7, 22]. The embedding vector of each item $c \in C$ is precomputed and denoted by n_c .
- (5) A recommendation similar to one that aligns with the user preference also aligns with the user preference. The similarity between two items can be measured by the Euclidean distance of their vector embeddings, and the similarity between two recommendations (of the same size) can be measured by the discrete earth mover's distance of the two sets of vector embeddings of the items. Generally, the earth mover's distance of two sets X and Y in the same metric space with the metric D is defined as $\text{EMD}(X, Y) := \min_{p \in \text{Bij}(X, Y)} \sum_{x \in X} D(x - p(x))$, where $\text{Bij}(X, Y)$ is the set of all bijections from X to Y .

The proposed framework will not work if the assumption (3) is not satisfied. It is advisable to verify this condition before implementing the framework.

3.1.2 High-level concept of the proposed method. Based on the assumptions (3) and (5) in Section 3.1.1, we adopt a strategy as follows: we first make the LLM give an instance of recommendation suited to the user's preference but not necessarily to the quantitative constraints, and then search for a constraint-abiding approximation to it using the proposed method. After all, a conversation between the user and the machine is carried out as follows (also described in Fig. 1).

- (1) Show the system's first message to facilitate conversation (such as instructions or a greeting).
- (2) Await a response from the user.
- (3) Give the whole conversation history to the LLM and receive its answer, using prompts to ensure that each name of the recommended items is enclosed with a marker pair (`<item>...</item>`).

- (4) Extract the set of the marked names (which are not necessarily in C) from the LLM's output and construct a feasible solution (constraint-abiding recommendation) which is similar to it (**draft** hereinafter) using the *correction algorithm* described later in Section 3.3. The draft and the corrected recommendation should be sets of the same size. That is, solve the minimization problem

$$\begin{aligned} & \text{minimize} && \text{EMD}(\{n_c \mid c \in X\}, N) \\ & \text{subject to} && X \subset C, |X| = |N|, \sum_{c \in X} v_c \in \Omega \end{aligned} \quad (1)$$

where N is the set of the vector embeddings of the marked items in the draft. The earth mover's distance is based on the elementwise Euclidean distance.

- (5) Replace the marked names of items in the LLM's answer with those from the corrected recommendation and show the corrected answer passage to users.
- (6) Go to (2) unless the user is satisfied with the result.

We denote the objective function of the problem (1) as $\text{DistToDraft}(X) := \text{EMD}(\{n_c \mid c \in X\}, N)$.

3.2 Overview of the algorithm

In order to realize the approximation part, we constructed an algorithm (hereinafter *correction algorithm*) based on a framework called *hill climbing*. Hill climbing is a heuristic approach to finding a local optimum instead of the global one. In this approach, you first take a random initial solution and then repeatedly apply partial changes to improve it gradually [23, 26]. There is also a chance to find a global optimum if the whole procedure of hill climbing is iterated multiple times. To complete this strategy, we implemented two routines: one to generate a random instance of recommendation that abides by the constraints, and another to perturbate an instance keeping its feasibility.

We show the landscape of the algorithm in Fig. 2. First, the algorithm finds an initial solution that satisfies the constraints. (Fig. 2 (1-A)) At this point, the solution is not setwise similar to the LLM's output, which means some elements do not approximate their counterparts in the draft recommendation well enough when you consider the optimal bijection between two sets. (Fig. 2 (1-B)) Those poor approximators are then replaced to make a partial change in the instance. Fig. 2 (2-A, 2-B) demonstrates constraint-abiding replacement of the three poorest approximators. This leads to improved setwise approximation, that is, smaller earth mover's distance. Repetition of these two procedures enables you to find an instance that approximates the recommendation instance given by the LLM while meeting the constraints. This heuristic is expected to be effective especially when it is almost impossible to find a constraint-abiding instance just by repeatedly determining all of the elements of the recommendation set randomly at once.

3.3 Correction Algorithm

The correction algorithm (Alg. 1) is a heuristic algorithm based on hill climbing to find a feasible solution "near" to an instance of LLM recommendation. We first describe the overview of the algorithm, and then expound upon its subroutines.

Here, m is the size of the draft and R_{trial} , R_{depth} and R_{width} are positive integer-valued hyperparameters. Subroutines `PICKONE`,

ITEMSTOREPLACE, and LOCALCHANGE and the method to compute the function DistToDraft are specified in the following sections.

Algorithm 1: Correction Algorithm

```

1 function CORRECTIONALGORITHM( $X$ )
2   Choose  $\vec{\beta} \in \Omega$  arbitrarily
3    $d \leftarrow \infty$ 
4   repeat  $R_{\text{trial}}$  times
5      $X \leftarrow \text{PICKONE}(\vec{\beta}, m)$ 
6     if  $d > \text{DistToDraft}(X)$  then
7        $d \leftarrow \text{DistToDraft}(X)$ ;  $X^* \leftarrow X$ 
8     repeat  $R_{\text{depth}}$  times
9        $I \leftarrow \text{ITEMSTOREPLACE}(X)$ 
10       $d' \leftarrow \infty$ 
11      repeat  $R_{\text{width}}$  times
12         $X' \leftarrow \text{LOCALCHANGE}(X, I)$ 
13        if  $d' > \text{DistToDraft}(X')$  then
14           $d' \leftarrow \text{DistToDraft}(X')$ ;  $X'^* \leftarrow X'$ 
15       $X \leftarrow X'^*$ 
16      if  $d > \text{DistToDraft}(X)$  then
17         $d \leftarrow \text{DistToDraft}(X)$ ;  $X^* \leftarrow X$ 
18  return  $X^*$ 

```

3.3.1 Random generation of a feasible solution. The proportion of feasible solutions to all subsets of C can be too small to allow runtime search, while the number of them is too large to allow pre-deployment enumeration. Therefore, we need to precompute and retain a data structure A from which we can potentially recover any feasible solutions at runtime.

The data structure A is $(d+1)$ -dimensional array of shape $(M_1 + 1) \times \dots \times (M_d + 1) \times (N + 1)$, where integers $M_1, \dots, M_d$ satisfy $\Omega \subset \{0, \dots, M_1\} \times \dots \times \{0, \dots, M_d\}$ and N is the upper bound of numbers expected to be sizes of solutions you want to recover. We use 0-based indexing, and a component $A[\beta_1, \dots, \beta_d, v]$ specified by a vector $\vec{\beta} = (\beta_1, \dots, \beta_d)$ and an integer v is briefly denoted by $A[\vec{\beta}; v]$. Each component $A[\vec{\beta}; v]$ is a Boolean value that represents the existence of a set $X \subset C$ that satisfies $\sum_{c \in X} v_c = \vec{\beta}, |X| = v$.

Suppose $C = \{c_1, \dots, c_{|C|}\}$ and define a $(d+1)$ -dimensional Boolean array B_i so that $B_i[\vec{\beta}; v]$ represents the existence of a set $X \subset \{c_1, \dots, c_i\}$ that satisfies $\sum_{c \in X} v_c = \vec{\beta}, |X| = v$. Then we have $B_{|C|} = A$ and A can be acquired by computing $B_1, B_2, \dots$ sequentially with overall time complexity $O(M_1 \dots M_d N |C|^2)$.

Based on the precomputed array A , the subroutine PICKONE shown in Alg. 2 finds a set $X \subset C$ which satisfies $\sum_{c \in X} v_c = \vec{\beta}, |X| = v$ if there is any. For a vector w , $w \geq 0$ means all entries of w are nonnegative.

Using this routine, you can generate potentially any set X that satisfies $\sum_{c \in X} v_c = \vec{\beta}, |X| = v$ (although not with equal probability).

3.3.2 Computation of the distance between a draft and a feasible solution. We defined the similarity between two sets of items in Section 3.1.1. Suppose N is the set of the vector embeddings of the marked items in the draft. For a given feasible solution

$X = \{x_1, \dots, x_{|N|}\}$ of size $|N|$ (the same as that of the draft), the distance between X and the draft is $\text{DistToDraft}(X) := \min_{p \in \text{Bij}(X, N)} \sum_{x \in X} |n_x - p(x)|$, where $\text{Bij}(X, N)$ is the set of all bijections from X to N . We also define a function OptPerm as $\text{OptPerm}(X) := \arg\min_{p \in \text{Bij}(X, N)} \sum_{x \in X} |n_x - p(x)|$.

The problem of finding the bijection p that minimizes $\sum_{i=1}^n C_{i,p(i)}$ for a given nonnegative matrix $C = (C_{i,j})$ of the size $n \times n$ is called the linear sum assignment problem (LSAP), which can be solved using an algorithm which has $O(n^3)$ time complexity [5]. Computation of DistToDraft(X) and OptPerm(X) is a special case of the LSAP with $(C_{i,j})_{1 \leq i, j \leq n}$ replaced by $(|n_x - y|)_{x, y \in X}$ and therefore takes $O(|N|^3)$ time.

3.3.3 Partial change of feasible solution. Some of the items in a feasible solution X may be close to the items to which they are mapped by OptPerm(X), while others may be distant from the corresponding items. The latter items can be viewed as preventing the entire solution from approximating the draft well, and are therefore suitable to be replaced at the next perturbation step in the hill climbing algorithm.

Here N denotes the set of the embedding vectors of the marked items in the draft and X a feasible solution of size m . Alg. 3 shows the subroutine to determine the subset I of X of the size 3 suitable to be replaced, named ITEMSTOREPLACE, and the subroutine to apply a partial change to X based on the choice of I , named LOCALCHANGE.

4 User Study: Diet Recommendation Under Nutritional Constraints

We applied the proposed method to a concrete problem to recommend a day's worth of meals through conversational interaction based on a catalog of dishes C based on Kikkoman Home Cooking¹, under nutritional constraints specified in Dietary Reference Intakes for Japanese [14]. OpenAI's model "gpt-3.5-turbo" is used

¹<https://www.kikkoman.co.jp/homecook/>

Algorithm 2: Subroutine PICKONE

```

1 function PICKONE( $\vec{\beta}, v$ )
2   loop
3      $\vec{\beta}' \leftarrow \vec{\beta}; v' \leftarrow v; L \leftarrow$  an empty list
4     while  $v' \neq 0$  do
5        $S \leftarrow \emptyset$ 
6       foreach  $c \in C$  do
7         if  $\vec{\beta}' - v_c \geq 0 \wedge A[\vec{\beta}' - v_c; v' - 1]$  then
8            $S \leftarrow S \cup \{c\}$ 
9       if  $S = \emptyset$  then
10        break
11      Choose  $c^* \in S$  at random
12       $\vec{\beta}' \leftarrow \vec{\beta}' - v_{c^*}; v' \leftarrow v' - 1$ ; append  $c^*$  to  $L$ 
13  if  $v' = 0$  and  $L$  has no duplicate then
14    return  $L$  as a set

```

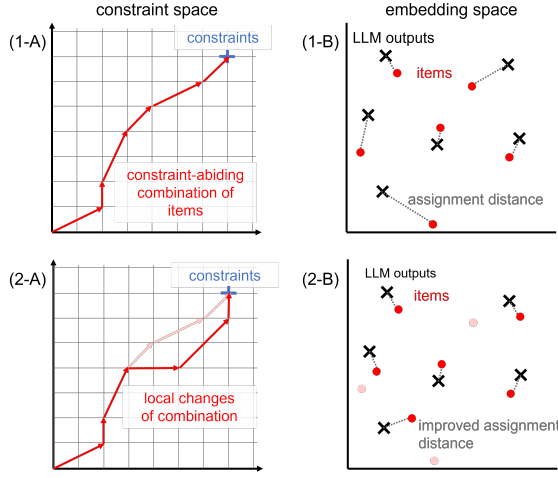


Figure 2: Concept of the proposed method

to generate the draft responses. The model “simcse-ja-bert-base-clmnp” available on Hugging Face ² is employed for computing vector embeddings.

The proposed method was implemented as a GUI application. In addition, the baseline method with the same interface was implemented. It replaced each recommended item with an existing item in a catalog and does not consider nutritional constraints, like in existing methods [8, 29]. Participants used both applications, and the user experience was investigated to evaluate the effectiveness of both systems.

A total of 24 employees from Preferred Networks, Inc. were recruited as the participants. Out of these, 16 were identified as male assigned at birth, and eight were identified as female assigned at birth. Furthermore, 13 participants were in their 20s, four were in their 30s, three were in their 40s, and one was in their 50s. At least one participant was a non-native Japanese speaker. Nine answered that they do not cook at all, eleven said they cook once to twice a

week, two people reported they cook three to four times a week, and two people stated they cook more frequently than that.

4.1 Task and Evaluation Measurements

The subjects were instructed to describe their preferences and requests regarding the daily menu through the conversation in the application in Japanese, and to explain in the reply what should be corrected if the latest recommendation is not suitable. Also, a subprogram in the application which is apart from the recommendation algorithm gave them information about whether the recommended set of dishes meets the nutritional conditions. The participants were told to try to guide the agent based on it so that the next recommendation would be adjusted to the constraints.

For the purpose of blinding, the GUI applications based on the proposed method and the baseline were presented as A and B, respectively. Half of the participants used A before using B, while the other half used B before using A. They continued the conversation until they were satisfied with the output or until they reached four input attempts. In order to determine the nutritional constraints, the participants were asked their ages and sexes assigned at birth as optional questions before or during the conversation. This was because the document [14] gives separate nutritional goals for males and females.

Furthermore, the participants were able to set a target caloric intake before or during the conversation, which was used to determine the vector of the desired total nutrients. In addition to the output text of the proposed method or the baseline, the subjects were presented with photographs of the recommended dishes, which were obtained from Kikkoman Home Cooking.

We checked whether the nutritional constraints were satisfied in each session. In the post-experiment survey, participants were asked six pairs of questions. Each pair is identical except that one is about the proposed method, and the other is about the baseline. Three pairs (Q1–Q3) were about the suitability of the recommended menus for the subject’s preference, while the remaining three pairs (Q4–Q6) were about the fulfillment of the nutritional requirements.

They were asked whether they were presented with a recommendation that fit their preferences on the first attempt (Q1-A, Q1-B) and after all (Q2-A, Q2-B). Participants were also asked to rate the level of effort required to guide the application so that the recommendation fit their preferences (Q3-A, Q3-B). Furthermore, we asked subjects whether they were presented with a recommendation that met the nutritional constraints on the first attempt (Q4-A, Q4-B) and after all (Q5-A, Q5-B). They were also asked to rate the level of effort required to guide the application so that the recommendation met the nutritional constraints (Q6-A, Q6-B). Participants answered these twelve questions on a 5-point Likert scale. The Wilcoxon signed-rank test was used to analyze the trend of the participants’ answers in the six aspects: Whether there was a difference in the trend of the answers to (T1) “Q1-A and Q2-A,” (T2) “Q1-B and Q2-B,” (T3) “Q1-A and Q1-B,” (T4) “Q2-A and Q2-B,” (T5) “Q3-A and Q3-B,” and (T6) “Q6-A and Q6-B.” The Bonferroni correction is utilized to counteract the multiple comparisons problem.

²https://huggingface.co/pkshatech/simcse-ja-bert-base-clmnp/blob/main/README_JA.md

Algorithm 3: Subroutine ITEMSToREPLACE, LOCALCHANGE

```

1 function ITEMSToREPLACE( $X$ )
2    $p \leftarrow \text{OptPerm}(X)$ 
3    $L \leftarrow X$  sorted by  $x \mapsto |n_x - p(x)|$  in descending order
4   return  $\{L[1], L[2], L[3]\}$ 
5 function LOCALCHANGE( $X, I$ )
6    $x_1, x_2, x_3 \leftarrow \text{elements of } I$ 
7    $x_4, \dots, x_m \leftarrow \text{elements of } X \setminus I$ 
8   loop
9      $Y \leftarrow \text{PICKONE}(v_{x_1} + v_{x_2} + v_{x_3}, 3)$ 
10     $y_1, y_2, y_3 \leftarrow \text{elements of } Y$ 
11    if  $\{y_1, y_2, y_3, x_4, \dots, x_m\}$  has no duplicate then
12      return  $\{y_1, y_2, y_3, x_4, \dots, x_m\}$ 

```

4.2 Result

In the sessions of the application A (the proposed method), 18 out of 99 instances of recommendation ended up raising errors (either the application did not generate a menu or it produced an output in the wrong format), while **all of the remaining 74 instances satisfied the imposed nutritional conditions**.

In the sessions of the application B (the baseline), seven out of 99 instances of recommendation ended up raising errors (either the application did not generate a menu or it produced an output in the wrong format). Furthermore, **all of the remaining 92 instances violated the imposed nutritional conditions**.

The results of the questionnaire is shown in Figure 3.

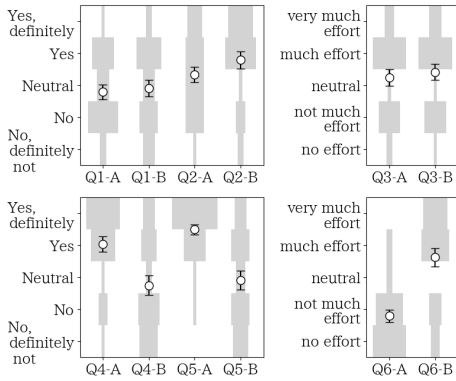


Figure 3: The feedback in the user study. Each of the twelve questions is specified in Section 4.1. The white circles represent the means. The black bars indicate the standard errors of the means. Each of the gray vertical histograms represents the relative frequencies of answers for each question.

In the following description of the results of the statistical test, the significance level instead of the p-values is modified by the Bonferroni correction. The significance level is set at $p = 0.05/6$.

- (T1) According to the one-tailed test where the alternative hypothesis is that the answers to Q2-A were more affirmative than those to Q1-A, the difference observed was marginally insignificant ($p = 0.0196 > 0.05/6$).
- (T2) According to the one-tailed test where the alternative hypothesis is that the answers to Q2-B were more affirmative than those to Q1-B, the null hypothesis was rejected ($p = 0.0006 < 0.05$).
- (T3) No significant difference was observed ($p = 0.6611$ in the two-sided test).
- (T4) No significant difference was observed ($p = 0.1915$ in the two-sided test).
- (T5) No significant difference was observed ($p = 0.5694$ in the two-sided test).
- (T6) According to the one-tailed test where the alternative hypothesis is that the answers to Q6-A were more apt to imply a lower level of effort, the null hypothesis was rejected ($p = 0.0005 < 0.05$).

5 Discussion

Based on the results of our study, the proposed system was able to recommend meal plans that satisfy individual nutrient constraints and frequently meet participants' preferences. We confirmed that the total nutrients of all recommended plans from the proposed system fit four types of nutrient constraints. In contrast, none of the meal plans recommended by the baseline system ever met all four nutritional constraints. This is unsurprising since no nutritional information is given to the LLM, which is inevitable due to the prompt length limit (16385 tokens for gpt-3.5-turbo). We also observed no significant performance difference concerning fulfilling user preferences between the proposed and the baseline system. Therefore, we argue that our proposed system can provide recommendations that meet constraints without severely losing the capacity of personalization compared to the baseline.

The proposed system occasionally did not satisfy participants' preferences in recommended meal plans. We speculate that there are two causes. First, draft plans generated from an LLM could fail to reflect participants' preferences (failure of the assumption (3)). When this happened, there is no reason for the final outputs from the proposed system to meet the preferences. One possible approach to this issue is to replace the language model with a more modern one, which can result in the generation of more tailored drafts that align with user preferences. Fine-tuning LLMs is another hopeful approach to providing responses specifically tailored to meal-planning recommendations. Second, preference reflection can deteriorate when we apply the correction algorithm if the proposed method cannot find appropriate items for the draft meal plans in the database. A possibility to improve the performance is that the method utilizes multi-modal similarity measures for recipe information (e.g., photos and ingredients) like previous studies [15, 19, 24]. Another possibility is that the performance could be improved by expanding the catalog with additional dishes.

References

- [1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (RecSys '23). ACM, New York, NY, USA, 1007–1014. <https://doi.org/10.1145/3604915.3608857>
- [2] J. Bijsterbosch and A. Volgenant. 2010. Solving the Rectangular assignment problem and applications. *Annals of Operations Research* 181 (2010), 443–462. <https://doi.org/10.1007/s10479-010-0757-3>
- [3] Joan Borràs, Antonio Moreno, and Aida Valls. 2014. Intelligent tourism recommender systems: A survey. *Expert systems with applications* 41, 16 (2014), 7370–7389. <https://doi.org/10.1016/j.eswa.2014.06.007>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] R.E. Burkard, M. Dell'Amico, and S. Martello. 2009. *Assignment Problems*. SIAM, Philadelphia, PA, USA. <https://doi.org/10.1137/1.9781611972238>
- [6] Dario Di Palma. 2023. Retrieval-augmented Recommender System: Enhancing Recommender Systems with Large Language Models. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (RecSys '23). ACM, New York, NY, USA, 1369–1373. <https://doi.org/10.1145/3604915.3608889>
- [7] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. *preprint* (2021). <https://doi.org/10.48550/arXiv.2104.08821>
- [8] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. 2023. Chat-REC: Towards Interactive and Explainable LLMs-Augmented Recommender System. *preprint* (2023). <https://doi.org/10.48550/arXiv.2303.14524>
- [9] Keita Higuchi, Hiroki Tsuchida, Eshed Ohn-Bar, Yoichi Sato, and Kris Kitani. 2020. Learning Context-dependent Personal Preferences for Adaptive Recommendation.

- ACM Trans. Interact. Intell. Syst. 10, 3, Article 23 (nov 2020), 26 pages. <https://doi.org/10.1145/3359755>
- [10] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning Vector-Quantized Item Representation for Transferable Sequential Recommenders. In *Proceedings of the ACM Web Conference 2023* (<conf-loc>, <city>Austin</city>, <state>TX</state>, <country>USA</country>, </conf-loc>) (WWW '23). ACM, New York, NY, USA, 1162–1171. <https://doi.org/10.1145/3543507.3583434>
 - [11] Yuki Koyama, Daisuke Sakamoto, and Takeo Igarashi. 2016. SelPh: Progressive Learning and Support of Manual Photo Color Enhancement. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) (CHI '16). ACM, New York, NY, USA, 2520–2532. <https://doi.org/10.1145/2858036.2858111>
 - [12] G. Linden, B. Smith, and J. York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing* 7, 1 (Jan 2003), 76–80. <https://doi.org/10.1109/MIC.2003.1167344>
 - [13] Annabel Laura Merrill and Bernice Kunerth Watt. 1973. *Energy value of foods: basis and derivation*. Human Nutrition Research Branch, Agricultural Research Service, United States Department of Agriculture.
 - [14] Ministry of Health, Labour and Welfare. 2019. Dietary Reference Intakes for Japanese (2020). Retrieved December 1st, 2023 from <https://www.mhlw.go.jp/content/10904750/000586553.pdf> (Japanese version) and <https://www.mhlw.go.jp/content/10900000/001150922.pdf> (English version).
 - [15] Taichi Nishimura, Katsuhiko Ishiguro, Keita Higuchi, and Masaaki Kotera. 2022. Multimodal Dish Pairing: Predicting Side Dishes to Serve with a Main Dish. In *Proceedings of the 1st International Workshop on Multimedia for Cooking, Eating, and Related Applications* (Lisboa, Portugal) (CEA++ '22). ACM, New York, NY, USA, 1–9. <https://doi.org/10.1145/3552485.3554934>
 - [16] Office for Resources Policy Division, Science and Technology Policy Bureau, Ministry of Education, Culture, Sports, Science and Technology (MEXT), Japan. 2015. Standard Tables of Food Composition in Japan - 2015 - (Seventh Revised Version). Retrieved December 1st, 2023 from https://www.mext.go.jp/a_menu/syokuhinseibun/1365297.htm (Japanese version) and https://www.mext.go.jp/en/policy/science_technology/policy/title01/detail01/1374030.htm (English version).
 - [17] OpenAI. 2023. GPT-4 Technical Report. *preprint* (2023). <https://doi.org/10.48550/arXiv.2303.08774>
 - [18] Sara Ozeki, Masaaki Kotera, Katsuhiko Ishiguro, Taichi Nishimura, and Keita Higuchi. 2022. Recipe Recommendation for Balancing Ingredient Preference and Daily Nutrients. In *The 1st International Workshop on Multimedia for Cooking, Eating, and related Applications*. ACM, New York, NY, USA, 11–19. <https://doi.org/10.1145/3552485.3554941>
 - [19] Dim P Papadopoulos, Enrique Mora, Nadiia Chepurko, Kuan Wei Huang, Ferda Ofli, and Antonio Torralba. 2022. Learning program representations for food images and cooking recipes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, New York, NY, USA, 16559–16569. <https://doi.org/10.1109/CVPR52688.2022.01606>
 - [20] Deuk Hee Park, Hyea Kyeong Kim, Il Young Choi, and Jae Kyeong Kim. 2012. A Literature Review and Classification of Recommender Systems Research. *Expert systems with applications* 39, 11 (2012), 10059–10072. <https://doi.org/10.1016/j.eswa.2012.02.038>
 - [21] Wenjun Peng, Derong Xu, Tong Xu, Jianjin Zhang, and Enhong Chen. 2023. Are GPT Embeddings Useful for Ads and Recommendation?. In *Knowledge Science, Engineering and Management*, Zhi Jin, Yuncheng Jiang, Robert Andrei Buchmann, Yaxin Bi, Ana-Maria Ghiran, and Wenjun Ma (Eds.). Springer Nature Switzerland, Cham, 151–162. https://doi.org/10.1007/978-3-031-40292-0_13
 - [22] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *preprint* (2019). <https://doi.org/10.48550/arXiv.1908.10084>
 - [23] Stuart J Russell. 2010. *Artificial Intelligence: A Modern Approach*. Pearson Education, Inc.
 - [24] Amaia Salvador, Erhan Gundogdu, Loris Bazzani, and Michael Donoser. 2021. Revamping cross-modal recipe retrieval with hierarchical transformers and self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, New York, NY, USA, 15475–15484. <https://doi.org/10.1109/CVPR46437.2021.01522>
 - [25] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based Collaborative Filtering Recommendation Algorithms. In *The 10th International Conference on World Wide Web* (Hong Kong, Hong Kong) (WWW '01). ACM, New York, NY, USA, 285–295. <https://doi.org/10.1145/371920.372071>
 - [26] Bart Selman and Carla P. Gomes. 2006. Hill-climbing search. In *Encyclopedia of cognitive science*. Vol. 81. John Wiley & Sons, Ltd, Hoboken, NJ, USA, 82.
 - [27] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *preprint* (2023). <https://doi.org/10.48550/arXiv.2312.11805>
 - [28] Johannes Von Oswald, Eyvind Niklasson, Ettore Randazzo, Joao Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. 2023. Transformers Learn In-Context by Gradient Descent. In *The 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 35151–35174.
 - [29] Likang Wu, Zhaopeng Qiu, Zhi Zheng, Hengshu Zhu, and Enhong Chen. 2023. Exploring Large Language Model for Graph Data Understanding in Online Job Recommendations. *preprint* (2023). <https://doi.org/10.48550/arXiv.2307.05722>
 - [30] Ying Xu, Tao Hu, and Ying Li. 2016. A travel route recommendation algorithm with personal preference. In *2016 12th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. IEEE, New York, NY, USA, 390–396. <https://doi.org/10.1109/FSKD.2016.7603205>
 - [31] Kazuyoshi Yoshii, Masataka Goto, Kazunori Komatani, Tetsuya Ogata, and Hiroshi G Okuno. 2008. An Efficient Hybrid Music Recommender System Using an Incrementally Trainable Probabilistic Generative Model. *IEEE Transactions on Audio, Speech, and Language Processing* 16, 2 (2008), 435–447. <https://doi.org/10.1109/TASL.2007.911503>
 - [32] Jizhi Zhang, Keqin Bao, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Is ChatGPT Fair for Recommendation? Evaluating Fairness in Large Language Model Recommendation. In *The 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (RecSys '23). ACM, New York, NY, USA, 993–999. <https://doi.org/10.1145/3604915.3608860>

A Example of the output of the proposed method

An example of the dialogue generated by the proposed method through interaction with a user during an experiment is shown in Table 1.

B Implementation Details of Diet Recommendation

B.1 Problem

A day's menu is a collection of dishes divided into three parts: breakfast, lunch, and dinner. Each dish should be chosen from dishes whose nutritional values are known since we need to check whether the menu meets the nutritional constraints. For this purpose, we stored data of 4,237 dishes obtained from Kikkoman Home Cooking, an online database where the dishes' names, recipes, photographs, and tables of ingredients are registered.

The output menu must meet the recommended intake of protein, fat, carbohydrates, and salt as specified in Dietary Reference Intakes for Japanese [14] as below. Note, however, that the proposed algorithm can also be used under nutritional conditions that vary from person to person, such as restrictions on carbohydrates or salt.

$$\alpha_1 \leq \frac{4[\text{kcal/g}] \times \text{protein}[\text{g}]}{\text{energy}[\text{kcal}]} \leq 0.2 \quad (2)$$

$$0.2 \leq \frac{9[\text{kcal/g}] \times \text{fat}[\text{g}]}{\text{energy}[\text{kcal}]} \leq 0.3 \quad (3)$$

$$0.5 \leq \frac{4[\text{kcal/g}] \times \text{carbohydrates}[\text{g}]}{\text{energy}[\text{kcal}]} \leq 0.65 \quad (4)$$

$$\text{salt}[\text{g}] \leq \alpha_2[\text{g}] \quad (5)$$

Here, the value of α_1 is 0.13 for people aged 15 to 49, 0.14 for people aged 50 to 64, and 0.15 for people aged 65 and above. The value of α_2 is 7.5 for "males" and 6.5 for "females". In practice, we set $\alpha_2 = 7.5$ for users who answered that they were assigned male at birth and $\alpha_2 = 6.5$ for those who answered otherwise or chose not to answer.

To be precise, the document [14] describes the conditions for protein, fat, and carbohydrates in terms of the ratio between energy produced by each nutrient and the total energy. Thus, to derive (2), (3), (4), we assumed some relation between the mass of nutrients and the amount of energy they produce. The Atwater factors (4, 9, and 4 kcal/g for protein, fat, and carbohydrates, respectively) are introduced in [13, 14] as coefficients conventionally used in cases like this. This study follows suit.

C' denotes the set of all recipes obtained from Kikkoman Home Cooking.

For each element (dish) $c \in C'$, $v_c \in \mathbb{Z}_{\geq 0}^d$ is its quantized nutritional values. First, the real-valued nutritional data were calculated based on the names and weight of ingredients fetched from Kikkoman Home Cooking and on the Standard Tables of Food Composition in Japan [16]. Then, the amount of protein, fat, carbohydrates, and salt was divided by 2g, 2g, 6g, and 0.5g, respectively, and rounded to the nearest integer to get $v_c \in \mathbb{Z}_{\geq 0}^d$ ($d = 4$).

Some recipes in Kikkoman Home Cooking serve more than one. In those cases, the amount of ingredients was divided by the number of people served.

It proved that there exist three dishes $c \in C'$ such that v_c is the zero vector. We defined C as the set obtained by removing those elements from C' and used it as a catalog.

Thus, this problem can be considered as a special case of the problem in Section 3.1.1, except for the following two differences.

- (Diff1) The constraints represented by v_c and Ω are not precisely what we want to meet since the v_c is not the exact amount of nutrients.
- (Diff2) Unlike in the general setting in Section 3.1.1, where the solution $X \subset C$ is simply a set, elements of the solution to this diet recommendation problem need to be divided into breakfast, lunch, and dinner.

B.2 Solution

We follow the solution given in Section 3 on the whole. In order to bridge the gaps pointed out above, two additional approaches are adopted.

To address the issue of inaccuracy in nutritional requirements due to quantization (Diff1), we add a verification step before every step to generate any set of recommendable items under approximate (quantized) constraints in the subroutines described in Section 3.3.1 and 3.3.3. In this additional step, it is verified whether the generated set meets the precise (not quantized) nutritional conditions, and if not, generation is repeated until they are complied with.

To deal with the necessity of the division of elements of the output into three meals (Diff2), we place the name of each dish output by the correction algorithm in the position of the corresponding item in the output passage of the LLM with regard to the bijection from the definition of the distance between recommendation sets as in Section 3.3.2. In addition, the LLM is nudged to output its draft recommendation in a format where items are distributed for breakfast, lunch, and dinner. In this way, the final output is guaranteed to be expressed as three meals.

In practice, when searching a correction menu of sizes 7, 8 or 9, it was always possible to find combinations that met nutritional requirements. However, it was often impossible to find solutions of size less than 7 or more than 9. Therefore, the program was set to raise an error when the draft contained fewer than 7 or more than 9 items.

OpenAI's model "gpt-3.5-turbo" is used to generate the system's responses in the conversation. The model "simcse-ja-bert-base-clmlp", which is available on Hugging Face, is employed for computing vector embeddings of names of dishes. These vectors are 768-dimensional and normalized.

The function `linear_sum_assignment` in the submodule `scipy.optimize` of SciPy is used for solving an LSAP. This function is based on the algorithm described in [2]. We set $R_{\text{trial}} = 6$, $R_{\text{depth}} = 8$, and $R_{\text{width}} = 1000$.

C Preliminary Experiment

A preliminary experiment is conducted to verify the assumption (3) in Section 3.1.1, the hypothesis underlying the proposed method:

Table 1: An instance of diet recommendation powered by the proposed method (translated from Japanese)

User input:	A healthy menu, say, low-fat Japanese cuisine (<i>target energy: 1557.5 kcal</i>)			
LLM output (draft):	Breakfast: <u>rice porridge with natto and spinach</u> , <u>tofu salad</u> Lunch: <u>salt-grilled mackerel</u> , <u>miso soup with shungiku and wakame seaweed</u> , <u>lightly-pickled cucumber</u> Dinner: <u>teriyaki chicken breast</u> , <u>Chinese-style stir-fried bean sprouts and cabbage</u> , <u>seaweed salad</u>			
Corrected output:	Breakfast: <u>white rice with green peas</u> , <u>tofu salad with vegetables</u> Lunch: <u>soused mackerel grilled with sesame and mirin</u> , <u>white rice mixed with shungiku</u> , <u>sakura shrimp stir-fried with carrot and paprika</u> Dinner: <u>dressed chicken tenderloin</u> , <u>chicken and vegetables simmered with consommé</u> , <u>vegetable salad nori wraps</u> Energy 1564.7 kcal Protein 65.8 g (appropriate*) Fat 39.7 g (appropriate*) Carbohydrates 220.2 g (appropriate*) Salt 4.9 g (appropriate*)			
User input:	I don't like fish, so I want the main dish to be changed			
LLM output (draft):	Breakfast: <u>natto rice</u> , <u>miso soup with vegetables and tofu</u> Lunch: <u>stir-fried chicken tenderloin with chili sauce</u> , <u>stir-fried eggplants with miso</u> , <u>spinach and mushrooms ohitashi</u> Dinner: <u>skewered chicken</u> , <u>boiled molokheiya with bonito flakes</u> , <u>white rice with edamame</u>			
Corrected output:	Breakfast: <u>avocado natto lettuce wraps</u> , <u>button mushrooms and vegetables simmered with tomato</u> Lunch: <u>bonito flakes and scrambled eggs rice balls</u> , <u>spicy stir-fried eggplants</u> , <u>spinach and mushrooms ohitashi</u> Dinner: <u>grilled chicken wrapped in nori</u> , <u>garnished chilled tofu</u> , <u>white rice with green peas</u> Energy 1544.7 kcal Protein 65.5 g (appropriate*) Fat 44.5 g (appropriate*) Carbohydrates 224.5 g (appropriate*) Salt 5.2 g (appropriate*)			

* Evaluation is based on the nutritional goal for females aged between 50 and 64.

the LLM is capable of providing preference-aligned (possibly hallucinatory) recommendations in response to the user's input.

To test the performance of the pure LLM, we designed a baseline menu recommendation system. This system is the same as that described in Section 3 except that the correction algorithm is replaced by a naive algorithm that only guarantees that items in the recommendation are all recommendable items and does not consider nutritional constraints, like in existing methods [8, 29]. This is realized by just replacing the parts of the LLM's output that are surrounded by markers (that is, the names of items to recommend) with the closest recommendable items with regard to the embedding vector distance one by one. We implemented this system as a command-line application and examined user experience.

Table 2 shows an example result of menu recommendation from the baseline method.

A total of eight employees from Preferred Networks, Inc. were recruited as the participants. Out of these, seven were identified as assigned male at birth, and one was identified as assigned female at birth. Seven were in their 20s, and one was in their 30s.

The subjects were instructed to describe their preferences and requests regarding the daily menu through conversation in Japanese on the application and to explain in the reply what should be corrected if the latest recommendation is not suitable.

Also, a subprogram in the application which is apart from the recommendation algorithm gave them information about whether the recommended set of dishes meets the nutritional conditions.

The participants were told to try to guide the agent based on it so that the next recommendation would be adjusted to the constraints. They continued the conversation until they were satisfied with the output or until they reached four input attempts.

In response to the question "Did you receive a recommendation of a menu that matched your preferences?" given after completing the task, two subjects answered "Yes, definitely," four people answered "Yes," one person answered "Neutral," and one person answered "No." No one answered "No, definitely not". In summary, six out of eight participants (75%) agreed that the recommendation given by the plain LLM fit their preferences.

On the other hand, despite the effort of the participants at correcting the recommendation in terms of nutrition, the program did not provide any instance of recommendation that meets nutritional conditions throughout this preliminary experiment.

From these results, we concluded that the assumption that the LLM has sufficient ability to adapt to user preferences was verified. Meanwhile, it was also demonstrated that the LLM cannot generate a recommendation that satisfies the user constraints by itself.

D Algorithm Evaluation

A questionnaire survey was conducted in order to evaluate the performance of the correction algorithm based on hill climbing compared to a baseline of the brute force search. The brute force

Table 2: An instance of conversation powered by the baseline method (translated from Japanese)

Input:	healthy dishes with lots of vegetables (<i>target energy: 2467.5 kcal</i>)					
Output:	Breakfast: <u>Bulgogi and rice with butter and soy sauce “temaki”, vegetable juice flavored with olive</u> Lunch: <u>green salad with watercress, soup with a lot of vegetables, wax gourd salad</u> Dinner: <u>baked vegetable salad,</u> <u>stir-fried and then boiled tofu and vegetables,</u> <u>vegetable curry with tomato puree</u> Energy 1911.4 kcal Protein 56.2 g (scarce*) Fat 88.4 g (excessive*) Carbohydrates 219.5 g (scarce*) Salt 19.9 g (excessive*)					
Input:	It should contain more protein and a bit less fat. I’d like more familiar vegetables rather than watercress, which is difficult to find.					
Output:	Breakfast: <u>grilled cod marinated in miso, natto salad</u> Lunch: <u>grilled chicken, salad with kiriboshi daikon,</u> <u>Japanese-style pumpkin soup</u> Dinner: <u>salmon sautéed in olive oil,</u> <u>spinach and scrambled eggs,</u> <u>starch-based soup with tofu and crab</u> Energy 1346.9 kcal Protein 86.6 g(excessive*) Fat 73.0 g(excessive*) Carbohydrates 77.3 g(scarce*) Salt 11.7 g(excessive*)					

*Evaluation is based on the nutritional goal for females aged between 50 and 64.

search was implemented using the CORRECTIONALGORITHM sub-routine with R_{depth} set to 0. In order to make the computation time of both approximately the same, we set $R_{\text{trial}} = 15000$.

D.1 Evaluation Method

We extracted eight pairs of user requests and corresponding draft diet plans generated by the LLM from the recorded results of the preliminary experiment (Section C). One of the drafts consisted of seven items, another of eight items, and the remaining six had nine items. We applied both the hill climbing and brute force algorithms to the drafts. We adopted the nutritional constraints for females aged 65 and above since this setting requires the most stringent constraints.

A total of 20 employees from Preferred Networks, Inc. were recruited as the participants. Out of these, 16 were identified as male and 4 were identified as female. One was in her 20s, eleven were in their 30s, three were in their 40s and two were in their 50s. Three did not disclosed their age.

In order to run the line 2 of Alg. 1 (“Choose $\vec{\beta} \in \Omega$ arbitrarily”) actually, you need to determine one ideal total nutrients. For this purpose, the desired total energy is specified to be 2375 kcal. It is the mean of the recommended value for males aged 30-49 of medium physical activity level and that for females with the same condition, according to [14].

Participants were presented with the user request, the LLM’s draft, and the algorithm outputs for all eight cases. Then, they were asked the following four questions and answered on a 5-point Likert scale. A total of 160 ($= 8 \times 20$) answers are acquired for each question.

- (q1) Which of the algorithm output satisfies the user request more?
- (q2) Which of the algorithm output approximates the LLM’s draft better?
- (q3-H) How desirable is the output of the hill climbing algorithm?

- (q3-B) How desirable is the output of the brute force algorithm?

Participants answered without knowing which of the outputs is generated by the hill climbing algorithm, as we did not disclose the algorithms used to produce the output.

D.2 Results

The average computation time was 3.6 seconds in the hill climbing case and 4.0 seconds in the brute force case, with a standard error of 0.09 and 0.13 second, respectively. This means the setting of the evaluation is not advantageous to the proposed method.

The average value of DistToDraft of algorithm outputs was 6.2 in the hill climbing case and 7.0 in the brute force case with a standard error of 0.28 and 0.29, respectively. This implies the hill climbing algorithm is effective to solve the optimization problem (1) in Section 3.1.2 in this case.

The distributions of the answers are shown in Fig. 4. In the feedback to (q1), 43.1% of all participants reported that the output of the hill climbing algorithm satisfies the user request more, while 11.3% answered the inverse. In the feedback to (q2), 42.5% of all participants reported that the output of the hill climbing algorithm approximates the LLM’s draft better, while 21.3% answered the inverse. In the feedback to (q3-H), 43.8% participants responded positively to the output of the hill climbing algorithm and 33.8% reacted negatively. In the feedback to (q3-B), 25.6% participants responded positively to the output of the baseline algorithm and 48.1% reacted negatively. Overall, the result of the questionnaire indicates that the hill climbing algorithm impressed participants as better with regard to the satisfaction of user requests, approximation of the draft, and simple desirability as a diet recommendation.

Overall, the results of this survey implied the advantage of the hill climbing approach in terms of both the quantitative criteria and user experience.

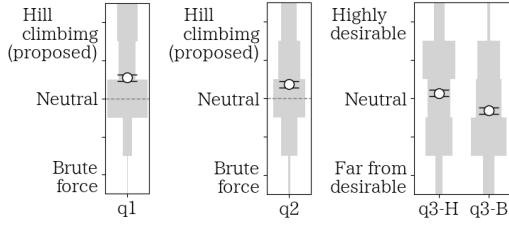


Figure 4: The feedback in the algorithm evaluation. The questions (q1), (q2), (q3-H), (q3-B) are specified in Section D.1. The white circles represent the means. The black bars indicate the standard errors of the means calculated as if all of the 160 answers were mutually independent. Each of the gray vertical histograms represents the relative frequencies of answers for each question.

E Qualitative Feedback in the User Study

One participant pointed out that the system was useful in that it allowed users to input their requests in natural language:

- “It seems nice that users can communicate their instruction through text entry, since that is easy for everybody.”

Some users pointed out that both methods tended to fail to catch user requests regarding a part of the recommendation compared to abstract and holistic requirements:

- “I have an impression that the system didn’t react to each word well, while it accepted abstract queries successfully.”
- “The system felt inconvenient to me since it often failed to reflect my requests which themselves seem easy to realize, such as ‘include noodles’ and ‘avoid the excessive content of salt.’ ”
- “I couldn’t control the system well when I wanted to replace specific dishes of the menu.”

A related phenomenon was observed in an instance of a conversation as follows:

the user requested that “tuna donburi with crispy Chinese yams” in the breakfast and “petit sushi roll of beef” in the dinner be removed from the recommendation. In response, the LLM substituted “stir-fried beef” for the dinner item “petit sushi roll of beef”, replaced all three breakfast items, including the “tuna donburi with crispy Chinese yams”, and internally outputted a new recommendation without changing the three lunch items. However, after applying the correction algorithm, the “tuna donburi with crispy Chinese yams” and “petit sushi roll of beef” appeared again in the breakfast and dinner, respectively, in the actual output.

F Additional Discussion

F.1 Issues with Interactivity

We observed some instances of conversation where the proposed method failed to reflect the users’ preference even after exchanging remarks. Moreover, in the baseline, we recognized a trend in the extent of adjustment to user preferences between the initial menu generated by a single prompt and the final menu after further conversation. However, this trend was not observed in the proposed

method. A possible explanation is that, although the LLM removed what should be removed according to the users’ explicit requests in the draft recommendation, the correction algorithm, which is intended to output a constraint-abiding recommendation in the proposed method, added the removed item to the menu again. This leads to inconsistency in the conversation and a lack of interactivity.

This problem in the proposed method may be solved by nudging the LLM to list items that should be excluded from the recommendation and then passing that information to a modified correction algorithm, which carries out optimization and avoids the banned items.

The interactivity can also be improved even when the output is incompatible with the user’s instruction by generating the explanation for why it was unmet, such as that it is impossible to include a requested deep-fried dish due to the restriction of fat content.

Also, inconsistencies between the menu and its description shown before and after it have been observed in some cases. This can be a natural result of our simplified scheme where names of dishes in the recommendation sentence are replaced based on the correction algorithm. One possible solution is to regenerate the description text based on the menu modified by the correction algorithm, although there is concern that this might result in longer response times of the application.

F.2 Validity as Catalog-based Recommendation

The proposed method has the capability of discovering menus that satisfy the nutritional constraints from a combinatorial number (more than 10^{27} in a typical case where the draft consists of nine dishes) of possible combinations of dishes in the catalog. Using them as initial values, it responsively constructs within the constraints recommendation, which approximates the LLM’s output by iteratively applying partial substitutions. Moreover, these constraints can vary from person to person, allowing personal restriction of fat or carbohydrates.

The proposed method can be applied to various constrained recommendation systems in other fields, such as e-shopping. In this case, the quantitative constraint here is the upper bound of the sum of prices and is one-dimensional. Since the lower dimension means a shorter computation time, a catalog of recommendable items of far larger size can be handled than in the case of diet recommendation while preserving responsiveness.

On the other hand, the generation of solutions that meet the constraints of Section 3.3.1 can be intractable where the dimension of constraints is higher. In fact, we could not construct a diet recommendation system that provides daily menus that satisfy the whole nutritional requirements defined in Dietary Reference Intakes for Japanese [14] that involves more than 30 nutrients, including vitamins and minerals.

Therefore, when applying this method to an application, the tradeoff between the desired level of responsiveness, the complexity of constraints, and the size of the catalog needs to be considered.

F.3 Limitations

The setting of the user study was partially culture-dependent and not necessarily universal. Given that all of the interactions were conducted in Japanese and the database of recommendable items

was created by a company in Japan, the implemented system was supposed to recommend diet plans based on Japanese culture. Moreover, since all the participants were working in Japan, the results of the questionnaire may reflect the Japanese values about food.

Nonetheless, the proposed framework itself is highly culture-independent and can be applied to dietary assistants or other agents potentially everywhere in the world. In particular, validating the performance of diet recommenders who speak other languages will be a promising future work.